

Módulo 5: Inteligência Computacional

Aplicada à Mineração de Dados

1 Objetivos de Aprendizagem

Ao final deste módulo, o estudante será capaz de:

- ✓ **Compreender** os fundamentos das Redes Neurais Artificiais, desde o neurônio de McCulloch-Pitts até arquiteturas modernas de Deep Learning.
- ✓ **Implementar** o algoritmo de retropropagação (backpropagation) e entender o papel das funções de ativação e otimizadores no treinamento.
- ✓ **Aplicar** técnicas de regularização (Dropout, BatchNorm, L1/L2) para evitar overfitting em redes neurais.
- ✓ **Explorar** arquiteturas de Deep Learning (CNN, RNN, LSTM, Transformers) e suas aplicações na mineração de dados.
- ✓ **Utilizar** algoritmos evolutivos (Algoritmos Genéticos, PSO) para otimização de hiperparâmetros e seleção de features.
- ✓ **Compreender** os princípios da Lógica Fuzzy e do Fuzzy C-Means para agrupamento com pertinência gradual.
- ✓ **Avaliar** e comparar abordagens de AutoML e Neural Architecture Search para automação do processo de modelagem.
- ✓ **Identificar** as tendências do estado da arte (2020–2024) em inteligência computacional aplicada à mineração de dados.

2 Redes Neurais Artificiais

2.1 Neurônio Biológico vs. Artificial

O sistema nervoso humano contém aproximadamente 86×10^9 neurônios, cada um conectado a milhares de outros por meio de sinapses. O neurônio biológico recebe sinais elétricos pelos **dendritos**, integra-os no **soma** (corpo celular) e, se o potencial de membrana ultrapassa um limiar, dispara um sinal pelo **axônio**.

Em 1943, **McCulloch & Pitts** propuseram o primeiro modelo matemático de neurônio artificial: um dispositivo binário que computa a soma ponderada das entradas e dispara (saída 1) se a soma excede um limiar θ :

$$y = \begin{cases} 1, & \sum_i w_i x_i \geq \theta \\ 0, & \text{caso contrário} \end{cases}$$

Em 1958, **Frank Rosenblatt** introduziu o **Perceptron**, adicionando pesos ajustáveis por aprendizado supervisionado. O modelo geral do neurônio artificial é:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) = f(\mathbf{w}^T \mathbf{x} + b)$$

onde $\mathbf{w} \in \mathbb{R}^n$ são os pesos, b é o viés (*bias*), e $f(\cdot)$ é a **função de ativação**.

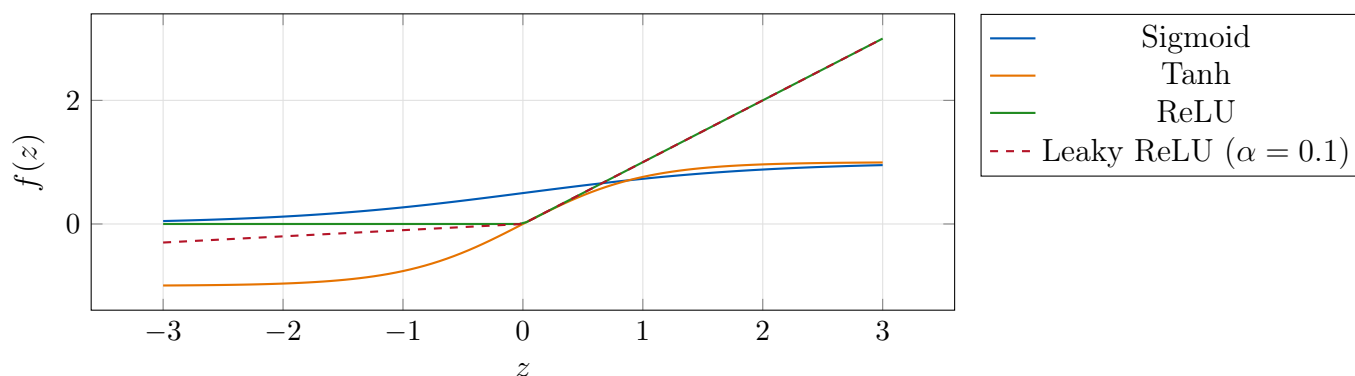
Neurônio Artificial

Um neurônio artificial é uma unidade computacional que realiza duas operações: (1) **pré-ativação** $z = \mathbf{w}^T \mathbf{x} + b$ (combinação linear) e (2) **ativação** $y = f(z)$ (não-linearidade). A composição de múltiplos neurônios em camadas forma uma Rede Neural Artificial (RNA).

2.2 Funções de Ativação

A não-linearidade introduzida pelas funções de ativação é o que permite às redes neurais aproximar funções arbitrariamente complexas. A Tabela 1 resume as principais funções.

A figura abaixo compara as formas das funções de ativação mais comuns:



Vanishing Gradient

Sigmoid e Tanh sofrem do problema do *vanishing gradient*: suas derivadas se aproximam de zero para valores extremos de z , fazendo com que o gradiente “desapareça” nas camadas iniciais durante o backpropagation. O ReLU mitiga esse problema, mas pode sofrer do *dying ReLU* (neurônios que sempre retornam zero). Leaky ReLU, ELU e GELU são alternativas robustas.

2.3 Perceptron Multicamadas (MLP)

Um **Perceptron Multicamadas** (MLP: *Multilayer Perceptron*) organiza neurônios em camadas: **entrada**, uma ou mais camadas **ocultas** e **saída**. Cada neurônio em uma camada está conectado a todos os neurônios da camada seguinte (*fully connected* ou *dense*).

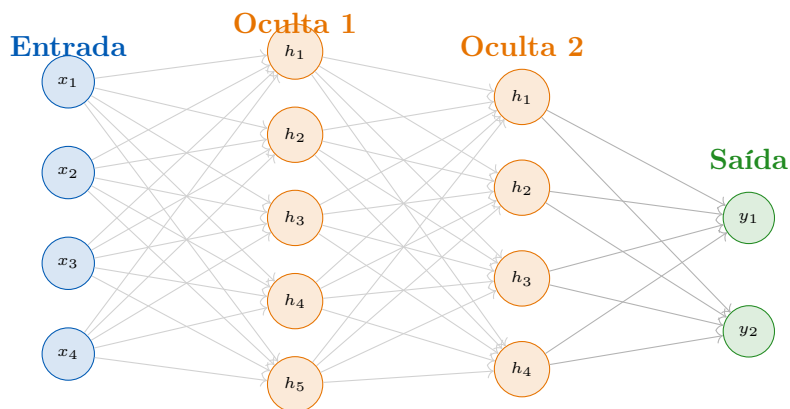
Tabela 1: Principais funções de ativação e suas propriedades.

Função	Fórmula	Intervalo	Propriedades	Uso Principal
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$(0, 1)$	Diferenciável; vanishing gradient	Saída binária
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Zero- centrada; vanishing gradient	Camadas ocultas
ReLU	$\max(0, z)$	$[0, +\infty)$	Eficiente; dying ReLU	Camadas ocultas (padrão)
Leaky ReLU	$\max(\alpha z, z), \alpha=0.01$	$(-\infty, +\infty)$	Evita dying ReLU	Alternativa ao ReLU
ELU	z se $z > 0$; $\alpha(e^z - 1)$ se $z \leq 0$	$(-\alpha, +\infty)$	Suave; out- puts negati- vos	Regularização im- plicita
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$; soma 1	Distribuição de probabili- dade	Saída multiclasse
GELU	$z \cdot \Phi(z)$	$\approx$ $(-0.17, +\infty)$	Suave; esto- cástica	Transformers, BERT

Notação: Para uma rede com L camadas, seja $\mathbf{a}^{(l)}$ a ativação da camada l , $\mathbf{W}^{(l)}$ a matriz de pesos e $\mathbf{b}^{(l)}$ o vetor de vieses:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)})$$

com $\mathbf{a}^{(0)} = \mathbf{x}$ (entrada) e $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ (saída).



i Teorema da Aproximação Universal

Hornik (1989): Uma rede neural com uma única camada oculta de tamanho suficiente e função de ativação não-linear pode aproximar qualquer função contínua em um subconjunto compacto de $\mathbb{R}^n$ com precisão arbitrária. Isso garante o poder expressivo das MLPs, mas não diz como treinar a rede ou quantos neurônios são necessários.

2.4 Algoritmo de Retropropagação (Backpropagation)

O **backpropagation** (Rumelhart, Hinton & Williams, 1986) é o algoritmo que viabilizou o treinamento de MLPs profundas, aplicando a **regra da cadeia** do cálculo para computar os gradientes da função de perda em relação a todos os pesos da rede.

Passo Forward: Dado um exemplo $(\mathbf{x}, y)$, computa-se as ativações camada a camada:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}), \quad l = 1, \dots, L$$

Função de Perda: Para regressão usa-se MSE; para classificação binária, cross-entropy:

$$\mathcal{L}(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

Passo Backward: Pelo teorema da cadeia, o gradiente em relação ao peso $w_{jk}^{(l)}$ é:

$$\frac{\partial \mathcal{L}}{\partial w_{jk}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(L)}} \cdots \frac{\partial z^{(l+1)}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w_{jk}^{(l)}}$$

Definindo o **erro local** $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \odot f'(\mathbf{z}^{(L)}), \quad \delta^{(l)} = (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \odot f'(\mathbf{z}^{(l)})$$

Atualização de pesos (gradiente descendente):

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \delta^{(l)} (\mathbf{a}^{(l-1)})^T, \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \delta^{(l)}$$

Algorithm 1: Backpropagation: Treinamento de MLP

Input: Dados $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$; taxa de aprendizado η ; épocas E
Output: Pesos $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$ e vieses $\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}$ treinados

- 1 Inicializar pesos aleatoriamente (ex. Xavier/He initialization);
- 2 **for** $\text{época} = 1$ **to** E **do**
- 3 Embaralhar os dados de treinamento;
- 4 **foreach** *mini-batch* $\mathcal{B} \subset \{1, \dots, N\}$ **do**
- 5 // Passo Forward
- 6 **for** $l = 1$ **to** L **do**
- 7 $\mathbf{z}^{(l)} \leftarrow \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$;
- 8 $\mathbf{a}^{(l)} \leftarrow f^{(l)}(\mathbf{z}^{(l)})$;
- 9 Calcular perda $\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(\hat{y}_i, y_i)$;
- 10 // Passo Backward
- 11 $\boldsymbol{\delta}^{(L)} \leftarrow \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot f'^{(L)}(\mathbf{z}^{(L)})$;
- 12 **for** $l = L - 1$ **to** 1 **do**
- 13 $\boldsymbol{\delta}^{(l)} \leftarrow (\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)} \odot f'^{(l)}(\mathbf{z}^{(l)})$;
- 14 // Atualização de parâmetros
- 15 **for** $l = 1$ **to** L **do**
- 16 $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \cdot \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T / |\mathcal{B}|$;
- 17 $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \cdot \boldsymbol{\delta}^{(l)} / |\mathcal{B}|$;

2.5 Otimizadores

A escolha do otimizador afeta significativamente a velocidade de convergência e a qualidade da solução encontrada.

SGD (Gradiente Descendente Estocástico):

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{g}_t$$

SGD com Momentum: Acumula a velocidade na direção do gradiente:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \mathbf{g}_t, \quad \mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{v}_t$$

Adam (Kingma & Ba, 2015): Combina Momentum e RMSProp com correção de viés:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1^\circ \text{ momento}) \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2^\circ \text{ momento}) \quad (2)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{correção de viés}) \quad (3)$$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \quad (4)$$

com $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Comparativo de otimizadores:

- **AdaGrad**: Adapta taxa de aprendizado por parâmetro; bom para dados esparsos; taxa decresce monotonamente.
- **RMSProp**: Variante do AdaGrad com janela deslizante; corrige o decaimento excessivo.
- **AdamW**: Adam com weight decay desacoplado (Loshchilov & Hutter, 2019); preferido em Transformers.

2.6 Regularização em Redes Neurais

Redes neurais profundas são propensas ao overfitting. As principais técnicas de regularização são:

- **Dropout** (Srivastava et al., 2014): Durante o treinamento, cada neurônio é desligado com probabilidade p (tipicamente $p = 0.5$). Funciona como uma média de ensemble implícita.
- **L2 Weight Decay**: Adiciona $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ à função de perda; incentiva pesos pequenos.
- **L1 Regularization**: Adiciona $\lambda \|\mathbf{w}\|_1$; incentiva esparsidade nos pesos.
- **Batch Normalization** (Ioffe & Szegedy, 2015): Normaliza as ativações em cada mini-batch:

$$\hat{z}^{(l)} = \frac{z^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \cdot \gamma + \beta$$

Acelera o treinamento e reduz a sensibilidade à inicialização.

- **Early Stopping**: Monitora a perda de validação; interrompe o treinamento quando ela começa a aumentar.
- **Data Augmentation**: Aumenta artificialmente o conjunto de treino (rotações, flips, ruído, etc.).

3 Deep Learning para Mineração de Dados

3.1 Redes Convolucionais (CNN)

As Redes Neurais Convolucionais (LeCun et al., 1998) exploram a **localidade** e a **invariância à translação** presentes em dados estruturados em grades (imagens, séries temporais).

Componentes principais:

- **Camada Convolutiva**: Aplica filtros $\mathbf{K}$ (kernels) deslizantes: $(f * \mathbf{K})(i, j) = \sum_m \sum_n f(i + m, j + n) \cdot \mathbf{K}(m, n)$. Extrai mapas de características locais.
- **Camada de Pooling**: Reduz a dimensionalidade (Max Pooling, Average Pooling). Introduce invariância.

- **Camada Fully Connected:** Classificador no topo da rede, após achatar (*flatten*) as features extraídas.

Aplicações em Mineração de Dados: Extração de features de imagens para classificação, análise de sentimentos com Conv1D sobre texto, detecção de anomalias em séries temporais com Conv1D, reconhecimento de padrões em EEG e sinais biomédicos.

3.2 Redes Recorrentes (RNN, LSTM, GRU)

Redes Recorrentes processam **sequências** mantendo um estado oculto $\mathbf{h}_t$ que captura dependências temporais:

$$\mathbf{h}_t = f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

O problema do *vanishing gradient* nas RNN básicas é resolvido pelas **Long Short-Term Memory (LSTM)** (Hochreiter & Schmidhuber, 1997) com quatro portões:

$$f_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{portão de esquecimento}) \quad (5)$$

$$i_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{portão de entrada}) \quad (6)$$

$$\tilde{C}_t = \tanh(\mathbf{W}_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad (\text{candidatos ao estado}) \quad (7)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{estado da célula}) \quad (8)$$

$$o_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{portão de saída}) \quad (9)$$

$$\mathbf{h}_t = o_t \odot \tanh(C_t) \quad (\text{saída}) \quad (10)$$

As **Gated Recurrent Units (GRU)** (Cho et al., 2014) simplificam a LSTM com apenas dois portões (atualização e reset), com desempenho comparável em muitas tarefas.

Aplicações: Predição de séries temporais, modelagem de linguagem, análise de sentimentos sequencial, detecção de anomalias temporais.

3.3 Attention e Transformers

O mecanismo de **Atenção** (Bahdanau et al., 2015) permite que o modelo pondere dinamicamente partes da entrada. O **Self-Attention** do Transformer (Vaswani et al., 2017) computa:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

onde Q (queries), K (keys) e V (values) são projeções lineares da entrada, e d_k é a dimensão das queries/keys (fator de escala).

Modelos pré-treinados relevantes para DM:

- **BERT** (Devlin et al., 2019): Encoder bidirecional; embeddings de texto de alta qualidade para classificação e busca.
- **GPT** (Radford et al., 2018): Decoder autoregressivo; geração de texto e few-shot learning.
- **TabNet** (Arik & Pfister, 2021): Arquitetura Transformer adaptada para dados tabulares; atenção sequencial em features; interpretável.

4 Computação Evolutiva

4.1 Algoritmos Genéticos

Inspirados na evolução biológica darwiniana, os **Algoritmos Genéticos** (Holland, 1975) mantêm uma população de soluções candidatas e as evoluem por operadores de seleção, crossover e mutação.

Componentes:

- **Cromossomo:** Codificação de uma solução candidata (binária, real, permutação).
- **População:** Conjunto de N cromossomos (tipicamente $N = 50-500$).
- **Função de Fitness:** Avalia a qualidade de cada indivíduo.
- **Seleção:** Escolhe pais; métodos: torneio, roleta, ranque.
- **Crossover:** Combina material genético de dois pais com probabilidade p_c (tipicamente 0.6–0.9).
- **Mutação:** Altera genes aleatoriamente com probabilidade p_m (tipicamente 0.001–0.01).
- **Elitismo:** Preserva os melhores indivíduos para a próxima geração.

Algorithm 2: Algoritmo Genético

Input: N (tamanho da população), G (gerações), p_c (prob. crossover), p_m (prob. mutação)

Output: Melhor solução encontrada

```

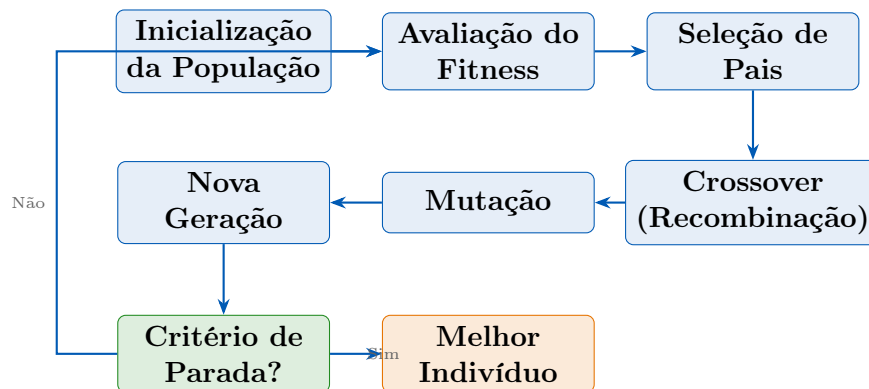
1  $P \leftarrow \text{InitializePopulation}(N)$  // cromossomos aleatórios
2  $F \leftarrow \text{EvaluateFitness}(P)$ ;
3 for  $g = 1$  to  $G$  do
4    $P_{\text{elite}} \leftarrow \text{SelectElite}(P, F)$ ;
5    $P' \leftarrow \emptyset$ ;
6   while  $|P'| < N - |P_{\text{elite}}|$  do
7      $p_1, p_2 \leftarrow \text{TournamentSelection}(P, F)$ ;
8     if  $\text{random}() < p_c$  then
9        $c_1, c_2 \leftarrow \text{Crossover}(p_1, p_2)$ ;
10    else
11       $c_1, c_2 \leftarrow p_1, p_2$ ;
12     $c_1 \leftarrow \text{Mutate}(c_1, p_m)$ ;  $c_2 \leftarrow \text{Mutate}(c_2, p_m)$ ;
13     $P' \leftarrow P' \cup \{c_1, c_2\}$ ;
14   $P \leftarrow P_{\text{elite}} \cup P'$ ;
15   $F \leftarrow \text{EvaluateFitness}(P)$ ;
16 return  $\arg \max_i F(P_i)$ ;

```

Aplicações em Mineração de Dados:

- **Seleção de Features:** Cromossomo binário indica quais features usar; fitness = acurácia em validação cruzada.

- **Otimização de Hiperparâmetros:** Evolui configurações de modelos (profundidade de árvore, taxa de aprendizado etc.).
- **Evolução de Regras:** Evolui conjuntos de regras de classificação (Pittsburgh approach).



4.2 Programação Genética

A **Programação Genética** (Koza, 1992) estende o AG evoluindo **programas** representados como **árvores de expressão**, ao invés de cromossomos de tamanho fixo.

Cada nó interno é um operador (+, −, ×, ÷, sin, log, ...) e cada folha é uma variável ou constante. O crossover troca subárvores entre indivíduos.

Aplicação principal: Regressão Simbólica: descobrir a equação matemática que melhor descreve os dados (ex. descobrir $y = 3x^2 + 2x - 1$ a partir de amostras ruidosas).

4.3 Particle Swarm Optimization (PSO)

O **PSO** (Kennedy & Eberhart, 1995) simula o comportamento coletivo de um bando de pássaros. Cada part cula i tem posi  o $\mathbf{x}_i$ e velocidade $\mathbf{v}_i$ no espa o de busca.

Equa  es de atualiza  o:

$$\mathbf{v}_{t+1} = \underbrace{\omega \mathbf{v}_t}_{\text{in rcia}} + \underbrace{c_1 r_1 (p_{best} - \mathbf{x}_t)}_{\text{cognitivo}} + \underbrace{c_2 r_2 (g_{best} - \mathbf{x}_t)}_{\text{social}} \quad (11)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1} \quad (12)$$

onde $\omega \approx 0.7$ (in rcia), $c_1 = c_2 \approx 1.5$ (acelera  es), $r_1, r_2 \sim U(0, 1)$, p_{best} = melhor posi  o da part cula, g_{best} = melhor posi  o do enxame.

Aplica  es em DM:

- Otimiza  o dos pesos de redes neurais.
- Busca de centr ides iniciais para K-Means.
- Otimiza  o de hiperpar metros de SVMs.
- Sele  o de features com codifica  o cont nuas.

5 Sistemas Fuzzy

5.1 Lógica Fuzzy

A **Lógica Fuzzy** (Zadeh, 1965) generaliza a lógica binária permitindo graus de pertinência $\mu_A(x) \in [0, 1]$ para conjuntos fuzzy. Um elemento pode pertencer parcialmente a um conjunto.

Funções de pertinência comuns:

$$\mu_{\text{tri}}(x; a, b, c) = \max\left(0, \min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right)\right) \quad (\text{triangular}) \quad (13)$$

$$\mu_{\text{trap}}(x; a, b, c, d) = \max\left(0, \min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right)\right) \quad (\text{trapezoidal}) \quad (14)$$

$$\mu_{\text{gauss}}(x; c, \sigma) = \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right) \quad (\text{Gaussiana}) \quad (15)$$

Variáveis Linguísticas: Permitem raciocinar com termos como “temperatura **alta**”, “pressão **média**”, onde cada termo é um conjunto fuzzy. Regras fuzzy: “*SE temperatura é alta E pressão é média, ENTÃO velocidade é baixa*”.

Sistema de Inferência Fuzzy (Mamdani ou Takagi-Sugeno): Fuzzificação $\rightarrow$ Avaliação das Regras $\rightarrow$ Agregação $\rightarrow$ Defuzzificação.

5.2 Fuzzy Mining

O **Fuzzy C-Means (FCM)** é uma extensão do K-Means onde cada ponto pode pertencer a múltiplos clusters com graus de pertinência. Minimiza a função objetivo:

$$J = \sum_{i=1}^N \sum_{k=1}^K u_{ik}^m \|x_i - c_k\|^2$$

onde $u_{ik} \in [0, 1]$ é o grau de pertinência do ponto x_i ao cluster k , $m > 1$ é o coeficiente de fuzzificação, e c_k são os centróides.

A atualização dos graus de pertinência é:

$$u_{ik} = \left(\sum_{j=1}^K \left(\frac{\|x_i - c_k\|}{\|x_i - c_j\|} \right)^{\frac{2}{m-1}} \right)^{-1}$$

e a atualização dos centróides:

$$c_k = \frac{\sum_{i=1}^N u_{ik}^m x_i}{\sum_{i=1}^N u_{ik}^m}$$

💡 Aplicação do FCM

Em segmentação de clientes, o FCM permite que um cliente pertença parcialmente a dois segmentos (ex. 60% “jovem digital” e 40% “econômico tradicional”), capturando melhor a realidade do que clusters exclusivos do K-Means.

6 AutoML e Neural Architecture Search

6.1 AutoML

AutoML automatiza o processo KDD, permitindo que não-especialistas construam modelos competitivos e acelerando o trabalho de cientistas de dados.

- **TPOT** (Olson et al., 2016): Usa Programação Genética para otimizar pipelines completos de ML (pré-processamento + seleção de modelo + hiperparâmetros). Representa pipelines como árvores.
- **Auto-sklearn** (Feurer et al., 2015): Combina **Otimização Bayesiana** (SMAC) com **meta-learning** (warm-starting com base em datasets similares) e **ensemble automático**. Vencedor de competições AutoML.
- **AutoGluon** (Erickson et al., 2020): Empilhamento (*stacking*) multicamadas automático de modelos; treina múltiplos modelos e os combina sem validação cruzada explícita, usando *bagging* intra-stack.

6.2 Neural Architecture Search (NAS)

O NAS automatiza o design de arquiteturas neurais, historicamente o gargalo mais custoso da DL.

- **DARTS** (Liu et al., 2019): Relaxa a busca discreta para um espaço contínuo, otimizando pesos de arquitetura e de rede simultaneamente por gradiente descendente.
- **EfficientNet** (Tan & Le, 2019): NAS para encontrar arquitetura base + *compound scaling* (balanceia profundidade, largura e resolução). State-of-the-art em eficiência.

6.3 Otimização de Hiperparâmetros (HPO)

- **Grid Search**: Busca exaustiva; $O(n^k)$; impraticável para muitos hiperparâmetros.
- **Random Search** (Bergstra & Bengio, 2012): Amostragem aleatória; melhor cobertura em alta dimensão; $O(k)$.
- **Otimização Bayesiana**: Usa surrogate model (GP ou TPE) para balancear exploração/exploração:
 - **Gaussian Process (GP)**: Preciso, caro para muitos parâmetros.
 - **Tree-structured Parzen Estimator (TPE)**: Estima $p(x|y > y^*)$ vs $p(x|y < y^*)$; eficiente.
- **Hyperband** (Li et al., 2017): Alocação sucessiva de recursos; para experimentos ruins cedo.
- **BOHB**: Combina Bayesian Optimization + Hyperband.
- **Optuna** (Akiba et al., 2019): Framework Python; suporta TPE, CMA-ES, pruning com MedianPruner.

7 Interação entre Técnicas

Sistemas de mineração modernos frequentemente **combinam** múltiplas técnicas, aproveitando as vantagens de cada abordagem:

- **CNN + GBM**: Extração de features com CNN pré-treinado (ex. ResNet) → classificação com XGBoost/LightGBM. Combina poder representacional do DL com eficiência dos GBMs.
- **NLP Embeddings + Clustering**: BERT gera embeddings de documentos → K-Means ou HDBSCAN para descoberta de tópicos. Supera LDA em qualidade de clustering.
- **GA + MLP + XGBoost**: GA realiza seleção de features → MLP e XGBoost são treinados com as features selecionadas → ensemble das predições.
- **Fuzzy + RNA**: Sistemas neuro-fuzzy (ex. ANFIS) combinam capacidade de aprendizado das RNAs com a interpretabilidade do raciocínio fuzzy.
- **PSO + Deep Learning**: PSO otimiza hiperparâmetros (taxa de aprendizado, dropout, número de camadas) da rede neural ao invés de busca em grade.

Pipeline Híbrido Recomendado

Para problemas de classificação em dados tabulares com features de alta dimensão: (1) Use GA ou SHAP para seleção de features, (2) aplique AutoML (Optuna + LightGBM) para baseline, (3) experimente TabNet ou FT-Transformer para comparação com DL, (4) combine os melhores modelos em ensemble por stacking.

8 Estado da Arte (2020–2024)

Transformers para Dados Tabulares

Arquiteturas Transformer, dominantes em NLP e visão, agora competem com GBDTs em dados tabulares:

- **FT-Transformer** (Gorishniy et al., 2021): Feature Tokenizer + Transformer; supera GBDTs em alguns benchmarks tabulares.
- **TabPFN** (Hollmann et al., 2022): Prior-Fitted Network treinado em datasets sintéticos; realiza inferência bayesiana aproximada; state-of-the-art em datasets pequenos.
- **SAINT** (Somepalli et al., 2021): Self-Attention em linhas e colunas; competitivo com LightGBM em benchmarks.
- O debate persiste: GBDTs ainda dominam em eficiência e interpretabilidade para a maioria dos datasets tabulares do mundo real.

Aprendizado por Representação (Self-Supervised)

Aprendizado auto-supervisionado elimina a necessidade de grandes conjuntos rotulados:

- **SimCLR** (Chen et al., 2020): Aprendizado contrastivo para imagens; usa augmentações como positivos e amostras distintas como negativos.
- **MoCo** (He et al., 2020): Momentum Contrast; eficiente com memória de queue de negativos.
- **SCARF** (Bahri et al., 2022): Adapta contrastive learning para dados tabulares; corrompe features aleatórias para criar visões; melhora fine-tuning com poucos rótulos.
- **Aplicação em DM**: Pré-treinamento em dados não rotulados + fine-tuning com poucos exemplos rotulados; muito útil em domínios médicos, financeiros.

Redes Generativas para Data Augmentation

Modelos generativos para criar dados sintéticos realistas:

- **GAN** (Goodfellow et al., 2014): Gerador vs. Discriminador em jogo minimax; gera amostras de alta qualidade.
- **CTGAN** (Xu et al., 2019): GAN condicional para dados tabulares; lida com distribuições mistas (contínuo + categórico); implementado no SDV framework.
- **TVAE** (Xu et al., 2019): Variational Autoencoder para dados tabulares; mais estável que GAN no treinamento.
- **Aplicações**: Balanceamento de classes desbalanceadas; geração de dados sintéticos para privacidade; aumentar datasets pequenos.

9 Exercícios

1. **Backpropagation Manual**: Dada uma rede com 2 entradas, 1 camada oculta com 2 neurônios (tanh) e 1 saída (sigmoid), com pesos iniciais $W^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.1 & 0.8 \end{pmatrix}$ e $W^{(2)} = \begin{pmatrix} 0.4 & -0.6 \end{pmatrix}$, realize um passo de backpropagation para o exemplo $(x_1 = 1, x_2 = 0, y = 1)$ com $\eta = 0.1$. Mostre todos os valores intermediários.
2. **Comparação de Funções de Ativação**: Explique por que o Sigmoid sofre de *vanishing gradient* mas o ReLU não. Em que situações o Leaky ReLU é preferível ao ReLU padrão? O que é o problema *dying ReLU*?
3. **Adam vs SGD**: Descreva as vantagens do otimizador Adam sobre o SGD simples.

Quando o SGD com momentum pode ser preferível ao Adam? Quais os hiperparâmetros do Adam e seus valores padrão recomendados?

4. **AG vs Busca em Grade:** Para otimização de hiperparâmetros de um modelo com 5 hiperparâmetros (3 valores cada), compare o custo computacional de Grid Search vs Algoritmo Genético com 20 gerações e população de 30 indivíduos. Qual abordagem encontra soluções melhores em espaços de alta dimensão? Justifique.
5. **Fuzzy C-Means:** Dado um conjunto com 4 pontos em $\mathbb{R}^1$: $x = \{1, 2, 8, 9\}$, aplique manualmente 2 iterações do FCM com $K = 2$, $m = 2$ e centróides iniciais $c_1 = 2$, $c_2 = 7$. Calcule os graus de pertinência u_{ik} e os novos centróides.
6. **Design de Arquitetura:** Para um problema de análise de sentimentos em tweets (texto de até 280 caracteres), proponha e justifique uma arquitetura de rede neural. Considere: representação do texto, tipo de rede, hiperparâmetros principais, estratégias de regularização.
7. **Comparação AutoML:** Pesquise os benchmarks OpenML-CC18 e AMLB (AutoML Benchmark). Compare TPOT, Auto-sklearn, AutoGluon e H2O AutoML em termos de acurácia média e tempo de execução. Quais as limitações de cada abordagem?

10 Referências

Referências

- [1] McCulloch, W. S.; Pitts, W. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.
- [2] Rosenblatt, F. The Perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [3] Rumelhart, D. E.; Hinton, G. E.; Williams, R. J. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [4] Hornik, K.; Stinchcombe, M.; White, H. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [5] LeCun, Y.; Bengio, Y.; Hinton, G. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [6] Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*. MIT Press, 2016.
- [7] Vaswani, A.; Shazeer, N.; Parmar, N.; et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [8] Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; Salakhutdinov, R. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

- [9] Ioffe, S.; Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning (ICML)*, 2015.
- [10] Kingma, D. P.; Ba, J. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
- [11] Arik, S. O.; Pfister, T. TabNet: Attentive interpretable tabular learning. *AAAI Conference on Artificial Intelligence*, 35(8):6679–6687, 2021.
- [12] Bahri, D.; Jiang, H.; Tay, Y.; Metzler, D. SCARF: Self-supervised contrastive learning using random feature corruption. *International Conference on Learning Representations (ICLR)*, 2022.
- [13] Holland, J. H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [14] Kennedy, J.; Eberhart, R. Particle swarm optimization. *Proceedings of IEEE International Conference on Neural Networks*, pp. 1942–1948, 1995.
- [15] Xu, L.; Skoularidou, M.; Cuesta-Infante, A.; Veeramachaneni, K. Modeling tabular data using conditional GAN. *Advances in Neural Information Processing Systems*, 32, 2019.